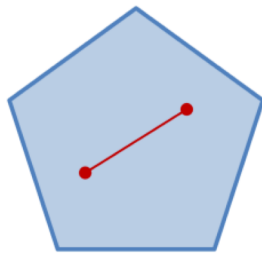
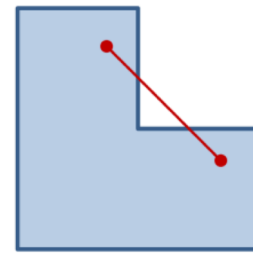


Convexity:



Convex

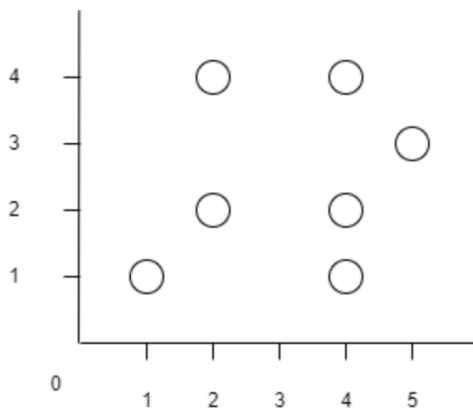


Concave

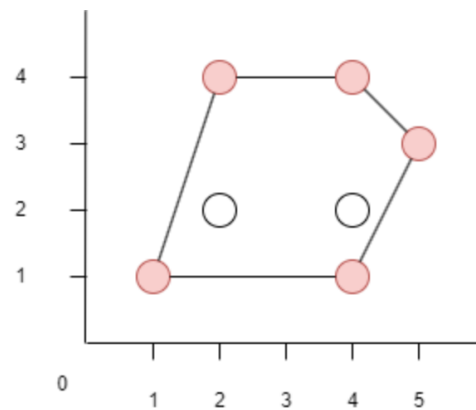
A shape is convex, if for any two points that are **inside** the shape, the line between these two points is also **inside** the shape.

Convex Hull:

Give a set of N input points, a convex hull is the smallest convex set containing all input points.



Points



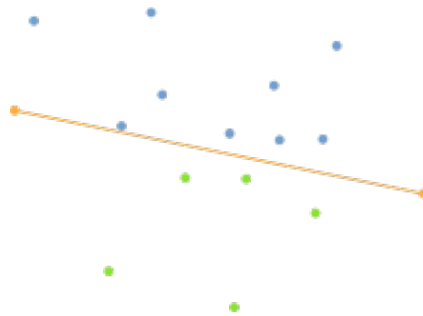
Its convex hull

Quick Hull Algorithm

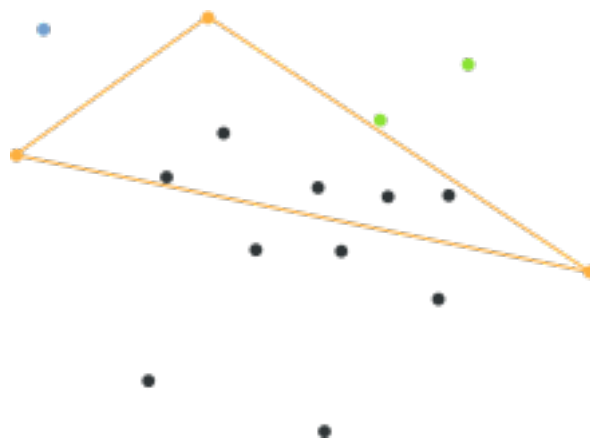
The 2-dimensional Quickhull algorithm can be broken down into the following steps:

- i. Find the points with minimum and maximum x coordinates, as these will always be part of the convex hull. If many points with the same minimum/maximum x exist, use the ones with the minimum/maximum y, respectively.
- ii. Use the line formed by the two points to divide the set into two subsets of points, which will be processed recursively. We next describe how to determine the part of the hull above the line; the part of the hull below the line can be determined similarly.
- iii. Determine the point above the line with the maximum distance from the line. This point forms a triangle with the two points on the line.
- iv. The points lying inside of that triangle cannot be part of the convex hull and can therefore be ignored in the next steps.

- v. Recursively repeat the previous two steps on the two lines formed by the two new sides of the triangle.
- vi. Continue until no more points are left, the recursion has come to an end and the points selected constitute the convex hull.



Steps 1-2: Divide the points into two subsets.:



Steps 3-5: Find a point with the maximum distance, ignore points inside the triangle, and recurse.

Side Calculation:

```
int findSide(iPair p1, iPair p2, iPair p)
{
    int val = (p.second - p1.second) * (p2.first - p1.first) -
              (p2.second - p1.second) * (p.first - p1.first);

    if (val > 0)
        return 1;
    if (val < 0)
        return -1;
    return 0;
}
```

This code uses the principle of the cross product of two 2D vectors to determine which side of a given line (formed by points p1 and p2) the point p lies on.

Here's the mathematical principle behind it:

Given two points $p1(x1, y1)$ and $p2(x2, y2)$ that form a line, and another point $p(x, y)$, we can create two vectors:

Vector v from $p1$ to $p2$: $(x2 - x1, y2 - y1)$

Vector u from $p1$ to p : $(x - x1, y - y1)$

The cross product of v and u is calculated as $(v.x * u.y) - (v.y * u.x)$. This is equivalent to $(p.second - p1.second) * (p2.first - p1.first) - (p2.second - p1.second) * (p.first - p1.first)$ in the given code.

Distance Calculation:

If the line passes through two points $P_1 = (x_1, y_1)$ and $P_2 = (x_2, y_2)$ then the distance of (x_0, y_0) from the line is:^[4]

$$\text{distance}(P_1, P_2, (x_0, y_0)) = \frac{|(x_2 - x_1)(y_1 - y_0) - (x_1 - x_0)(y_2 - y_1)|}{\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}}.$$